

Particle Filter Localization with Lidar

Description

This project implements a particle filter for robot localization in a mapped indoor environment. The robot state consists of planar position and heading, and the filter uses control inputs together with noisy lidar range measurements to estimate the robot pose over time. The particle filter represents the state distribution using weighted samples, making it well suited for nonlinear systems and non-Gaussian uncertainty.

Problem Formulation

The robot state is defined as

$$x = \begin{bmatrix} p_x \\ p_y \\ \theta \end{bmatrix}$$

where p_x and p_y are the 2D position coordinates and θ is the heading angle.

The input consists of the commanded speed s and heading rate ω , while the measurement is a scalar lidar range to the nearest wall. Ground-truth states, inputs, and measurements are provided from the recorded data. The goal is to estimate the robot pose over time using a particle filter.

System Model

The continuous-time dynamics are given by

$$\begin{aligned} \dot{p}_x &= (s + \nu_s) \cos(\theta) \\ \dot{p}_y &= (s + \nu_s) \sin(\theta) \\ \dot{\theta} &= \omega + \nu_\omega \end{aligned}$$

where ν_s and ν_ω are process noise terms.

Using forward Euler integration with sampling period T , the discrete-time model becomes

$$\begin{aligned} p_{x,k+1} &= p_{x,k} + (s_k + \nu_{s,k}) \cos(\theta_k) T \\ p_{y,k+1} &= p_{y,k} + (s_k + \nu_{s,k}) \sin(\theta_k) T \\ \theta_{k+1} &= \theta_k + (\omega_k + \nu_{\omega,k}) T \end{aligned}$$

The lidar measurement model is nonlinear and is evaluated using the provided map-based function $h(x, k)$.

Initial Conditions

We define the following covariance matrices

$$\begin{aligned} P_0 &= \begin{bmatrix} \sigma_x^2 & 0 & 0 \\ 0 & \sigma_y^2 & 0 \\ 0 & 0 & \sigma_\theta^2 \end{bmatrix} & Q &= \begin{bmatrix} \sigma_s^2 & 0 \\ 0 & \sigma_\omega^2 \end{bmatrix} \\ R &= \sigma_r^2 & \sigma_r &= 10cm \\ \sigma_x &= 0.1cm & \sigma_s &= 32cm/s \\ \sigma_y &= 0.1cm & \sigma_\omega &= 0.55rad/s \\ \sigma_a &= 5 \times 10^{-5}rad \end{aligned}$$

Where P_0 is the initial state covariance matrix, Q is the process noise covariance matrix, and R is the measurement noise. Each σ^2 is the covariance term for its respective state/input/measurement.

Particle Filter Method

The state distribution is represented by a set of particles and associated weights. The particles are initialized from a Gaussian distribution with mean equal to the initial true state and covariance determined by the assumed uncertainty in position and heading.

At each time step, the filter performs the following steps:

Update: The particle weights are updated using the likelihood of the lidar measurement under a Gaussian measurement noise model.

Estimate: The state estimate is computed as the weighted average of the particles, and the state covariance is estimated from the weighted particle deviations.

Resample: When the effective number of particles becomes too small, resampling is performed to eliminate low-weight particles and duplicate high-weight particles.

Propagate: The particles are advanced through the nonlinear dynamics using the measured inputs and independently sampled process noise.

Simulation

The particle filter is evaluated under four different initialization conditions:

1. Known starting position and known heading (10 particles)
2. Unknown starting position and known heading (25 particles)
3. Known starting position and unknown heading (40 particles)
4. Unknown starting position and unknown heading (650 particles)

When the starting position is known, the initial position standard deviations are set to nominal values. When the starting position is unknown, the initial standard deviations in x and y are increased to 100 cm. When the starting heading is unknown, the initial heading standard deviation is set to π rad.

These cases illustrate how the required number of particles increases as the initial uncertainty becomes larger.

Results

Path Plots

Figures 1–4 show the true and estimated robot trajectories for the four initialization cases.

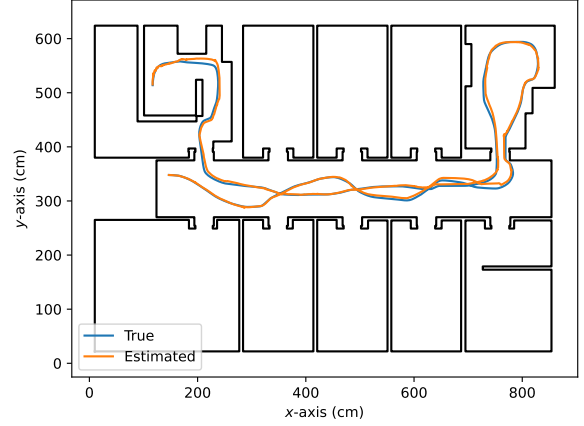


Figure 1: Known starting position and known heading.

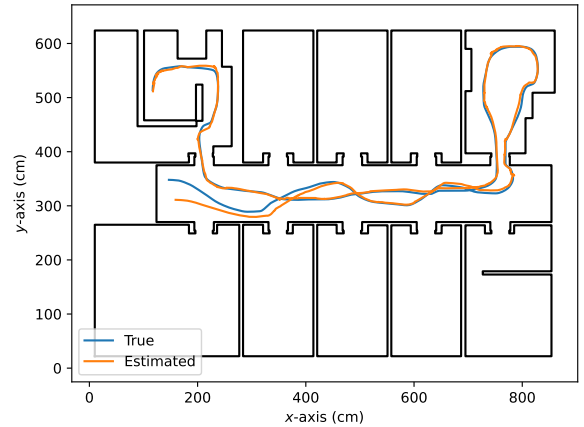


Figure 2: Unknown starting position and known heading.

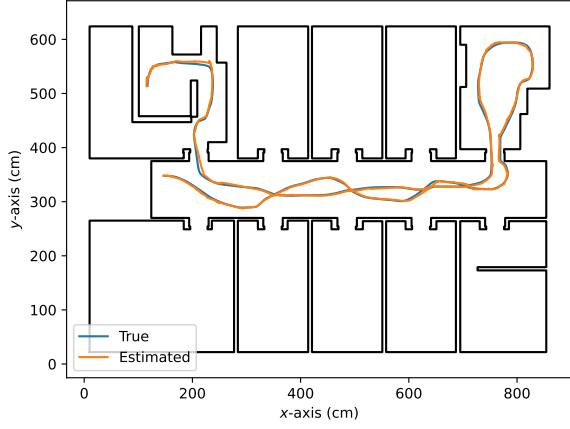


Figure 3: Known starting position and unknown heading.

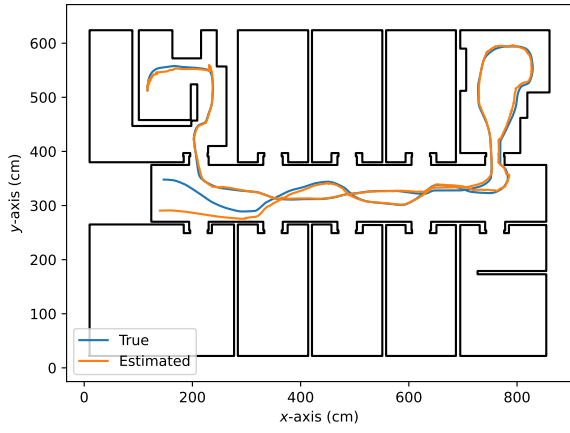


Figure 4: Unknown starting position and unknown heading.

Error Plots

For the cold-start case, Figures 5–7 show the estimation errors for position and heading.

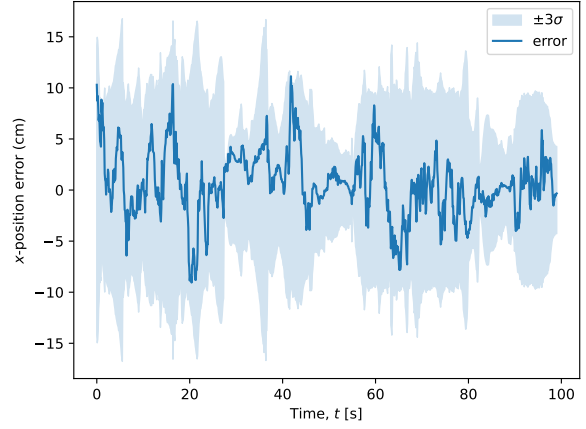


Figure 5: x -position estimation error.

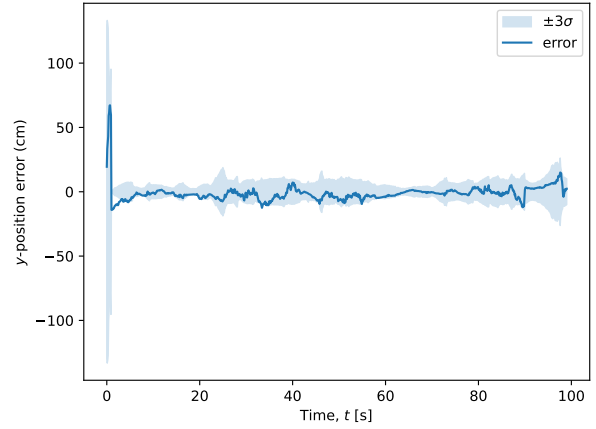


Figure 6: y -position estimation error.

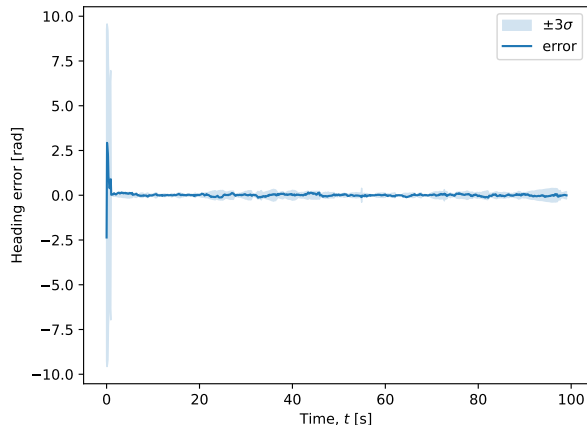


Figure 7: Heading estimation error.

Observations

Starting with the two cases that are the most intuitive, the first case, with a known starting position and known heading performed very well, even with just 10 particles. This is because it is quite easy to track an object with low noise, even with only 10 particles.

The second intuitive case is the fourth case, with both an unknown starting position and an unknown heading. The filter performed well overall, but took the longest to converge on the truth. With a large region of position uncertainty and a large range of heading possibilities, it takes a large number of particles to encapsulate the truth. However, after the measurements come in, the particles closest to truth will begin to be weighted much higher and the filter will quickly converge (assuming good measurements).

The cases that are fairly unintuitive are cases 2 and 3, where only one of the position and heading is unknown. Case 2, with an unknown position and a known heading, consistently produced the highest error. Taking a longer amount of time to converge, on average, than case 4 seems counterintuitive, but diving deeper makes sense. The initial uncertainty of the robot's position is just as large as case 4, but with far fewer particles. As a result, the particles are spread too thin to find the robot's true location. The filter then latches onto incorrect position estimates, which in turn corrupt the "known" heading estimate as the filter tries to explain measurement errors.

The last case is case 3, with a known position and unknown heading. I originally thought this would be

a higher error case, but it turned out to be the lowest on average. I underestimated the power of adding more particles, and 40 turned out to be more than enough to cover the range of possible headings. With the position known, the lidar measurements provide very clear information, allowing the filter to quickly and accurately determine the correct heading. However, as you decrease the number of particles, this case causes a divergence of the filter quite often.

Most of what we discussed above was about the initial conditions of the particle filter and how long each took to converge. One important thing to note is that after convergence, the filters with more particles estimated the true position of the robot more accurately. This seems fairly self-explanatory.

On the topic of resampling, it is also important to note the effective sample values I found to be most accurate. I ended up keeping $Je = 0.6$, so if the effective number of samples drops below 0.6, I would resample. This number seems reasonable in that it is not too high that we let the particles spread out too much, and not too low that we don't let them spread out enough. On average, this number looked to provide the highest accuracy in each case. Lowering or increasing the number significantly caused the filter to diverge frequently, as expected.

Discussion

The results show that particle filters can successfully localize a robot in a nonlinear environment using noisy control inputs and lidar measurements. When the initial uncertainty is small, relatively few particles are needed because the filter only needs to represent a narrow region of the state space. As the initial uncertainty increases, many more particles are required to adequately cover the possible robot poses.

From the animation, the particles tended to spread out in regions where the lidar measurements were less informative or where multiple poses produced similar expected measurements. The particles coalesced when the robot encountered geometrically informative parts of the environment, allowing the measurement update to strongly favor a smaller subset of poses.

This behavior highlights one of the main strengths of the particle filter: it can represent multi-hypothesis uncertainty and then concentrate around the correct solution as informative measurements become available.